WEBRUM-08: Dashboards and Alerting

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 8 of 10 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

An effective RUM monitoring strategy requires both visibility (dashboards) and proactive detection (alerting). Dashboards provide at-a-glance health views for different audiences — executives care about business impact and Apdex trends, while operations teams need real-time error rates and performance breakdowns.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with at least 7 days of data
Permissions	`storage:events:read`, `storage:metrics:read`, dashboard create permissions
Previous Notebooks	All prior WEBRUM notebooks (01-07)

1. Executive RUM Dashboard KPIs

Executive dashboards should be simple, business-focused, and actionable. Key metrics:

KPI	What It Tells Executives	Target
Apdex score	Overall user satisfaction (0-1)	> 0.85
Session count	Traffic volume and trends	Depends on business
Error rate %	Percentage of sessions with errors	< 2%
Bounce rate %	Users leaving after one page	< 40%
CWV pass rate	% of page loads meeting all 3 CWV	> 75%
Avg session duration	User engagement level	Varies by app type

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; primary_tags.application
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.waiting_duration
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the "real_user" literal itself could NOT be confirmed here and is the
documented value form;
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Executive KPI summary — single-query dashboard tile
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize
    total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0),
    bounce_sessions = countIf(user_action_count == 1),
    avg_actions = avg(user_action_count),
    avg_duration_min = avg(duration / 1m),
    by:{primary_tags.application}
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    bounce_rate_pct = round(toDouble(bounce_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    avg_duration_min = round(avg_duration_min, decimals: 1)
| sort total_sessions desc
```

```
// Session volume trend — 7-day daily session counts
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| makeTimeseries session_count = count(), interval:1d, by:
{primary_tags.application}
```

2. Apdex Score Calculation

The **Application Performance Index (Apdex)** is an open standard for measuring user satisfaction. It classifies every user action into three buckets based on a threshold T:

Classification	Duration	Description
Satisfied	≤ T	Action completed within acceptable time
Tolerating	T < duration ≤ 4T	Action was slow but user waited
Frustrated	> 4T	Action was too slow or failed

Apdex formula:

```
Apdex = (Satisfied + (Tolerating / 2)) / Total
```

The threshold T is configurable per application. Common defaults:

Action Type	Default T
Page load	3 seconds
XHR action	2.5 seconds
Route change	2.5 seconds

```
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**, but was written
// against names that are null on New RUM data, so these cells returned nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0, javascript agent):
//   action.type == "Load"             -&gt; characteristics.classifier == "page_summary"
//   action.type                       -&gt; user_action.type (hard_navigation | same_view)
//   action.name                       -&gt; page.detected_name
//   web_vitals.largest_contentful_paint-&gt; lcp.start_time       (327,099 populated)
//   web_vitals.cumulative_layout_shift -&gt; cls.value            (387,254 populated)
//   app.name                          -&gt; dt.rum.application.id
// UNITS CHANGE WITH THE FIELD. web_vitals.* was a nanosecond DURATION, so `/ 1ms` was correct for
// it; lcp.start_time is a PLAIN NUMBER already in milliseconds, and dividing it by 1ms yields
// null. Compare it against the 2500/4000 ms thresholds directly.
// `web_vitals.*` does still exist in the same schema, but carried 16 records in 30 days against
// lcp.*'s 327,099 — it is not a different RUM generation, just a rarely-populated sibling.
// Apdex calculation with T = 3 seconds for page loads
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_sec = duration / 1s
| summarize total = count(),
    satisfied = countIf(duration_sec &lt;= 3),
    tolerating = countIf(duration_sec &gt; 3 and duration_sec &lt;= 12),
    frustrated = countIf(duration_sec &gt; 12),
    by:{primary_tags.application}
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0) / toDouble(total), decimals: 3)
| sort apdex asc
```

```
// Apdex trend over 7 days — daily Apdex score
fetch user.events, from:-7d
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_sec = duration / 1s
| fieldsAdd is_satisfied = if(duration_sec &lt;= 3, 1.0, else: 0.0),
    is_tolerating = if(duration_sec &gt; 3 and duration_sec &lt;= 12, 0.5,
else: 0.0)
| makeTimeseries
    total_actions = count(),
    satisfied_score = sum(is_satisfied),
    tolerating_score = sum(is_tolerating),
    interval:1d
```

```
// Apdex by page — which pages have the worst user satisfaction?
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_sec = duration / 1s
| summarize total = count(),
    satisfied = countIf(duration_sec &lt;= 3),
    tolerating = countIf(duration_sec &gt; 3 and duration_sec &lt;= 12),
    by:{page.detected_name}
| filter total &gt; 20
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0) /
toDouble(total), decimals: 3)
| sort apdex asc
| limit 10
```

Interpreting Apdex Scores

Apdex Range	Rating	Action
0.94 - 1.00	Excellent	No action needed
0.85 - 0.93	Good	Monitor for changes
0.70 - 0.84	Fair	Investigate slow pages
0.50 - 0.69	Poor	Prioritize optimization
< 0.50	Unacceptable	Immediate attention required

3. Operational RUM Dashboard

Operations teams need real-time visibility into errors, performance anomalies, and traffic patterns.

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type == "RouteChange" -&gt; "same_view"  (the New RUM SPA
route-change value; the
//                              only other value is "hard_navigation".
"Custom" has NO equivalent.)
//   connection.type         -&gt; network.protocol.name
// CLASSIFIER MATTERS AS MUCH AS THE FIELD: navigation-timing fields
(performance.dom_interactive,
// performance.load_event_end) live on classifier "navigation" and are 0 on
"page_summary", so a
// page_summary filter silently empties them. ttfb.* is the opposite — it
lives on page_summary.
// Real-time error rate — errors per 15 minutes over the last 6 hours
fetch user.events, from:-6h
| filter characteristics.has_error == true
| makeTimeseries error_count = count(), interval:15m, by:{error.type}
```

```
// Active error summary — current top errors in the last hour
fetch user.events, from:-1h
| filter characteristics.has_error == true
| summarize error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{error.reason, error.type, primary_tags.application}
| sort affected_sessions desc
| limit 10
```

```
// Performance SLA — percentage of page loads under 3 seconds
fetch user.events, from:-1h
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_sec = duration / 1s
| summarize total = count(),
    under_3s = countIf(duration_sec &lt;= 3),
    by:{primary_tags.application}
| fieldsAdd sla_pct = round(toDouble(under_3s) / toDouble(total) * 100.0,
decimals: 1)
| sort sla_pct asc
```

4. RUM Alerting Strategies

Effective RUM alerting requires the right balance — too sensitive triggers alert fatigue, too loose misses real issues.

Recommended Alert Categories

Category	What to Alert On	Threshold Example
Error rate spike	Sudden increase in JS error rate	> 5% of sessions with errors
Apdex drop	User satisfaction below threshold	Apdex < 0.7 for 15 minutes
Performance degradation	p75 page load exceeds baseline	p75 > 5s for 15 minutes
Traffic anomaly	Unexpected drop in session volume	< 50% of previous day's hourly average
CWV regression	Core Web Vitals crossing into Poor	LCP p75 > 4s for 30 minutes

Alert Configuration

Modern path (recommended for new alerting): build these conditions as **Davis anomaly detectors** (`builtin:davis.anomaly-detectors`), configured in the Anomaly Detection app and driven by DQL. Detectors read Grail, so they work against the New RUM field vocabulary the rest of this series uses, and they carry forward past the upgrade.

Legacy path — metric events: the classic surface below still works and is what most existing RUM alerting uses, but `builtin:anomaly-detection.metric-events` is flagged **Blocked at upgrade**, and the `builtin:apps.web.*` metric keys it alerts on are Metrics Classic keys that **DQL cannot query** (FAQ entry 11 §3.1). Both facts point the same way: do not author new RUM alerting here.

Configure the legacy path via:

Settings > Anomaly detection > Metric events > Add metric event

Setting	Recommended Value
Metric	RUM metric (e.g., `builtin:apps.web.action.apdex`)
Evaluation window	15 minutes (balances speed and noise)
Sliding window	5-minute intervals
Threshold type	Static or auto-adaptive baseline

5. Error Rate Alerts

The following queries provide the data foundation for error rate alerting. Use these patterns to detect when error rates exceed acceptable thresholds.

```
// Error rate per 15-minute window — alert threshold data
fetch user.sessions, from:-6h
| filter dt.rum.user_type == "real_user"
| fieldsAdd has_error = if(error.count &gt; 0, 1.0, else: 0.0)
| makeTimeseries
    total_sessions = count(),
    error_sessions = sum(has_error),
    interval:15m,
    by:{primary_tags.application}
```

```
// New errors — errors first seen in the last getHour(potential deployment
issue)
fetch user.events, from:-1h
| filter characteristics.has_error == true
| summarize first_seen = min(timestamp),
    error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{error.reason, primary_tags.application}
| sort first_seen desc
| limit 10
```

6. Performance Degradation Alerts

Detect when page load performance degrades beyond acceptable thresholds.

```
// p75 page load duration per 15-minute window — performance alert data
fetch user.events, from:-6h
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_ms = duration / 1ms
| makeTimeseries p75_duration = percentile(duration_ms, 75), interval:15m, by:
{primary_tags.application}
```

```
// Apdex per 15-minute window — satisfaction alert data
fetch user.events, from:-6h
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_sec = duration / 1s
| fieldsAdd apdex_score = if(duration_sec &lt;= 3, 1.0,
    else: if(duration_sec &lt;= 12, 0.5,
    else: 0.0))
| makeTimeseries avg_apdex = avg(apdex_score), interval:15m, by:
{primary_tags.application}
```

> **Tip:** For production alerting, use Dynatrace Intelligence anomaly detection rather than static thresholds. Dynatrace Intelligence automatically baselines normal behavior and detects deviations, reducing false positives from seasonal traffic patterns.

7. RUM + Synthetic Combined View

RUM and synthetic monitoring complement each other. Combining them provides a complete UX monitoring picture:

Aspect	RUM Covers	Synthetic Covers
Availability	Only when users visit	24/7 scheduled monitoring
Baseline	Real-world conditions (variable)	Clean-room conditions (stable)
Coverage	Pages users visit	All critical paths (scripted)
Error detection	Errors users hit	Errors in scripted flows
Geographic	Where users are	From configured locations

Side-by-Side Comparison

Use `append` to compare RUM and synthetic performance for the same application:

```
// RUM session summary — real user experience
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize rum_sessions = count(),
    rum_error_sessions = countIf(error.count &gt; 0),
    rum_avg_actions = avg(user_action_count),
    by:{primary_tags.application}
| fieldsAdd monitoring_type = "RUM",
    error_rate_pct = round(toDouble(rum_error_sessions) /
toDouble(rum_sessions) * 100.0, decimals: 1)
```

```
// Synthetic vs RUM — compare session types for the same app
fetch user.sessions, from:-24h
| summarize session_count = count(),
    avg_duration_sec = avg(duration / 1s),
    error_sessions = countIf(error.count &gt; 0),
    by:{primary_tags.application, dt.rum.user_type}
| filter dt.rum.user_type == "real_user" or dt.rum.user_type == "SYNTHETIC"
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(session_count) * 100.0, decimals: 1)
| sort primary_tags.application asc, dt.rum.user_type asc
```

8. Summary and Series Recap

In this notebook, we covered:

- **Executive KPIs** — Session count, error rate, bounce rate, Apdex
- **Apdex calculation** — Satisfied/tolerating/frustrated classification via DQL
- **Operational dashboard** — Real-time error rates, SLA tracking, active errors
- **Alerting strategies** — Error rate spikes, Apdex drops, performance degradation
- **RUM + Synthetic** — Combining both monitoring types for complete coverage

WEBRUM Series Recap

Notebook	Topic	Key Takeaway
01	RUM Fundamentals	JavaScript agent, data model, basic queries
02	SPA Instrumentation	Route changes, framework tips, XHR monitoring
03	Core Web Vitals	LCP, INP, CLS measurement and scoring
04	Session Analysis	Segmentation, journeys, conversions, bounce rates
05	Error Analysis	Error impact, rage clicks, error-session correlation
06	Performance Analysis	Waterfall, TTFB, slow pages, geographic performance
07	Session Replay	Privacy, masking, finding and correlating replays
08	Dashboards & Alerting	Apdex, KPI dashboards, alerting strategies

References

- Apdex Standard
- Dynatrace RUM Dashboards
- Metric Events for Alerting
